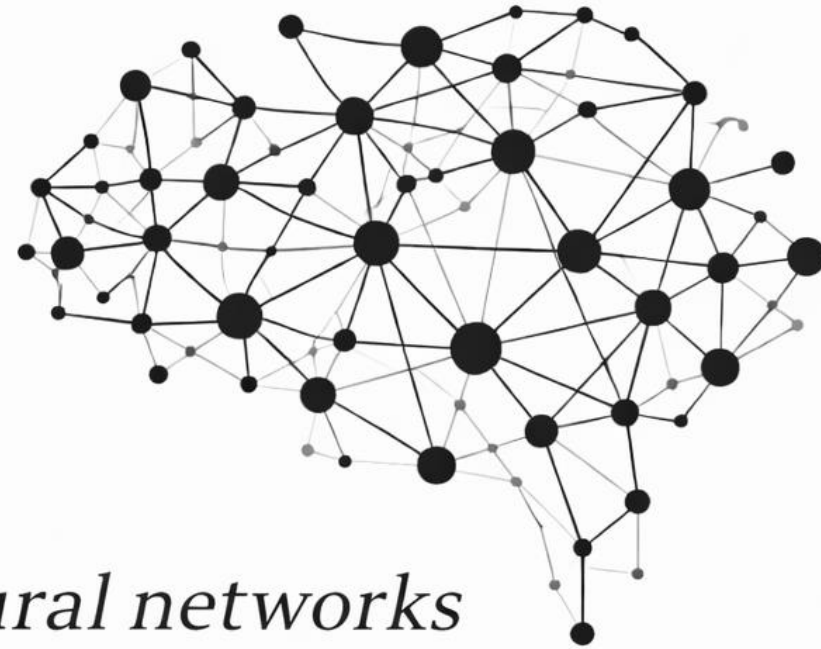
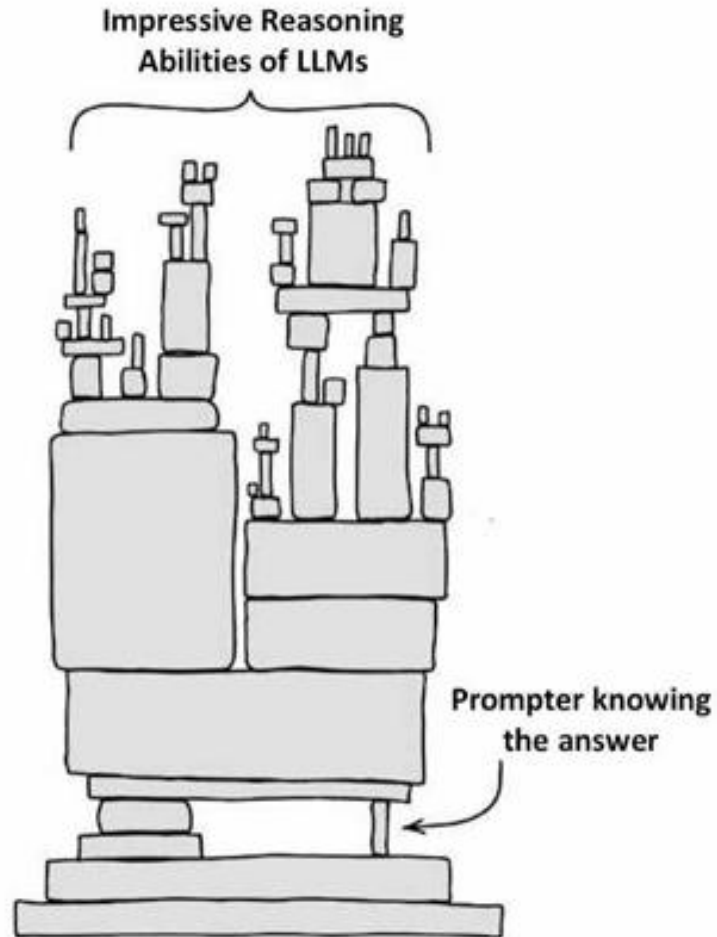


Neural Network



*“ The true power of neural networks
is not in how they are built, but in
the endless possibilities they create. ”*

Our Future !

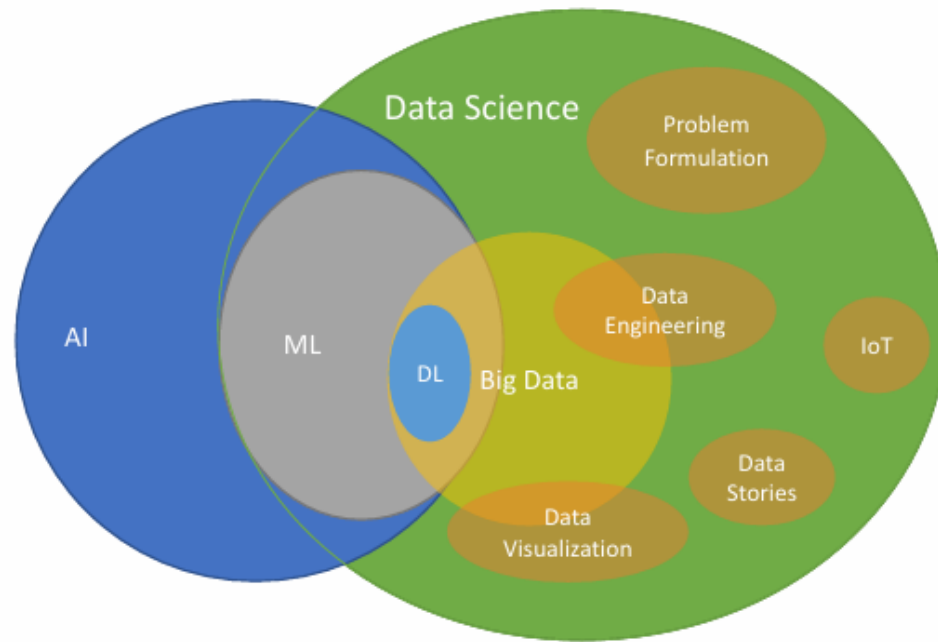


- Large Language Models demonstrate impressive reasoning abilities
- Prompting helps extract knowledge from models
- Human–AI collaboration will dominate future workflows

Learning Objectives

- Understand **what a Neural Network is**
 - Explain **neurons, layers, and models**
 - Differentiate between **parameters and hyper-parameters**
 - Describe the role of a **loss function**
 - Understand **optimization algorithms** (e.g., Gradient Descent)
 - Explain the **Development–Testing paradigm**
 - Define **iteration, batch, and epoch**
 - **Hands-on** “*Linear & Logistic Regression with Single Neuron*”
-
- Understand **NLP (Natural Language Processing), NLU (Understanding) and NLG (Generation)**
 - Identify the **major steps in an NLP application**
 - Explain **tokenization**
 - Understand **sequence models** and where they are used
 - **Hands-on** “*Text Vectorization & Word Embedding Experiment*”

Data Science : ML/AI/DL



- Data Science is an umbrella term
- It is the full building that we showed
- It has foundation, pillars, floors, walls, interiors, maintenance
- One can focus on a part of the building and develop deep expertise
- But should know the breadth as well

Types of Data

- Tabular Data
 - Most common form
 - Arises in almost all business use cases
 - Usually number of data points x features
- Timeseries Data
 - Tabular but at different times (a logical ordering in time)
- Image Data
 - Increasing in recent years
 - Usually number of data points x height x width x sensor channels
- Video Data
 - Time series of image data is video data
 - Vision Tasks
- Text Data
 - Language tasks
 - Usually text corpus – Needs to be converted to number – How?
- Audio Data
 - Language tasks
 - Usually recording corpus – Signal Processing

Academic Performance & Personalized Learning

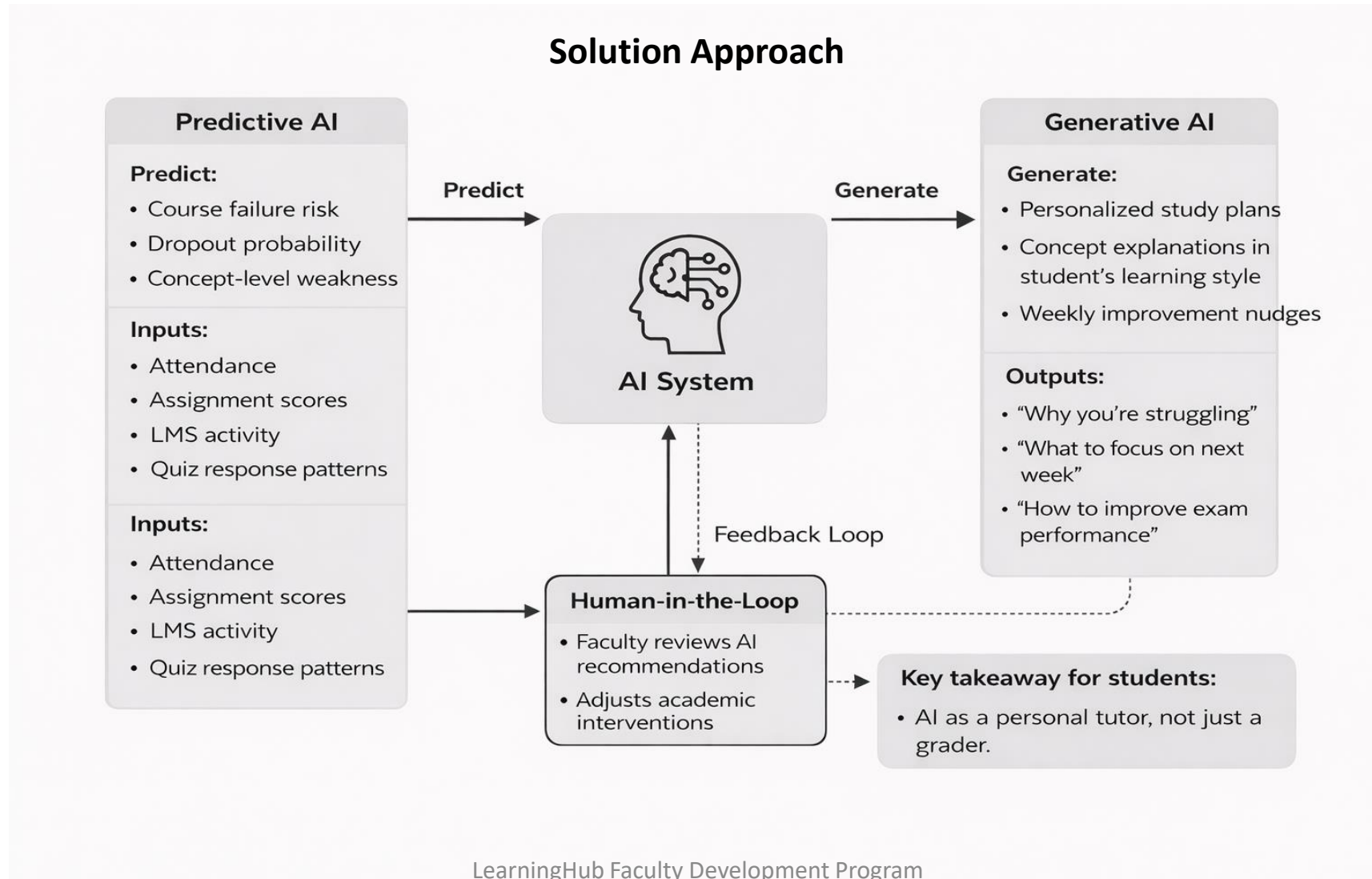
Problem Statement

“Traditional education treats all students the same, despite vast differences in learning pace, background, and cognitive strengths.

Faculty struggle to identify early academic risk, while students receive feedback too late to improve outcomes.”



Academic Performance & Personalized Learning



Predictive AI and Generative AI

Predictive AI

- Input: Any of the data modality
- Output: Continuous or Categorical

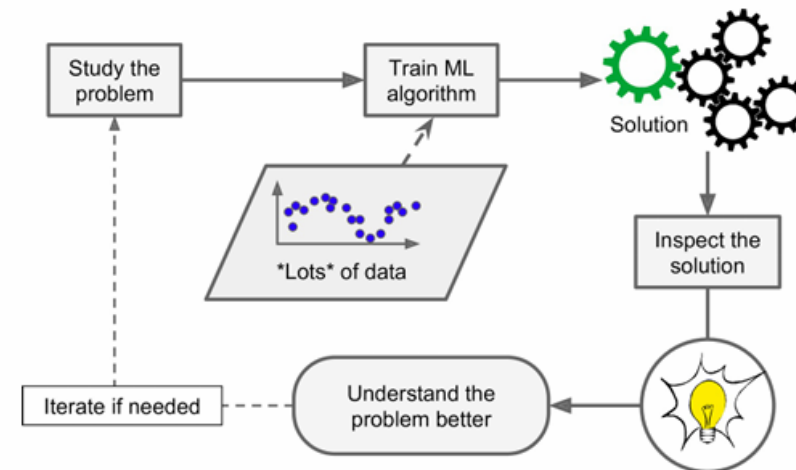
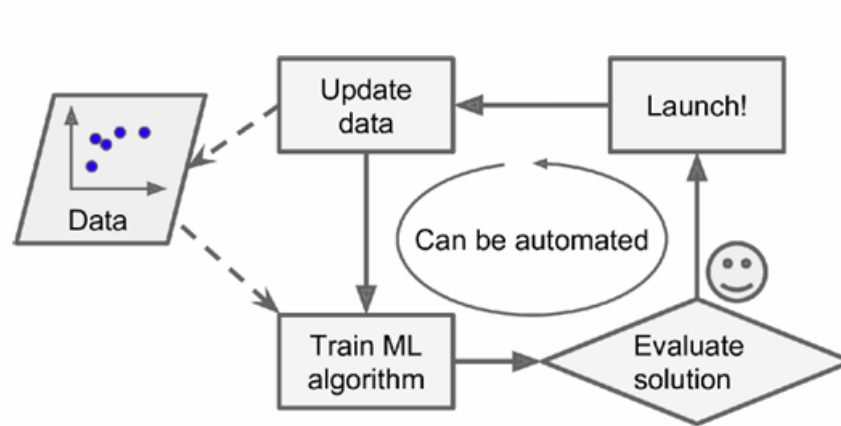
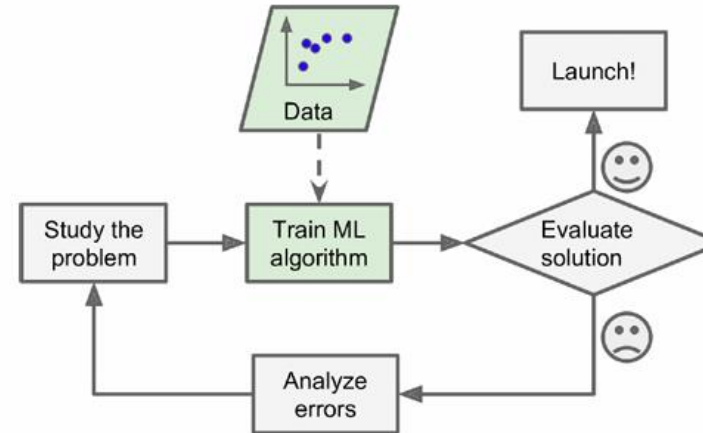
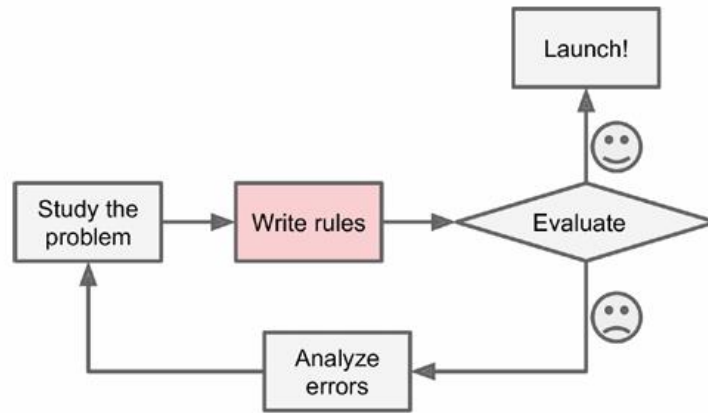
Generative AI

- Input: Any of the data modality
- Output: Text, Image, Video, Audio

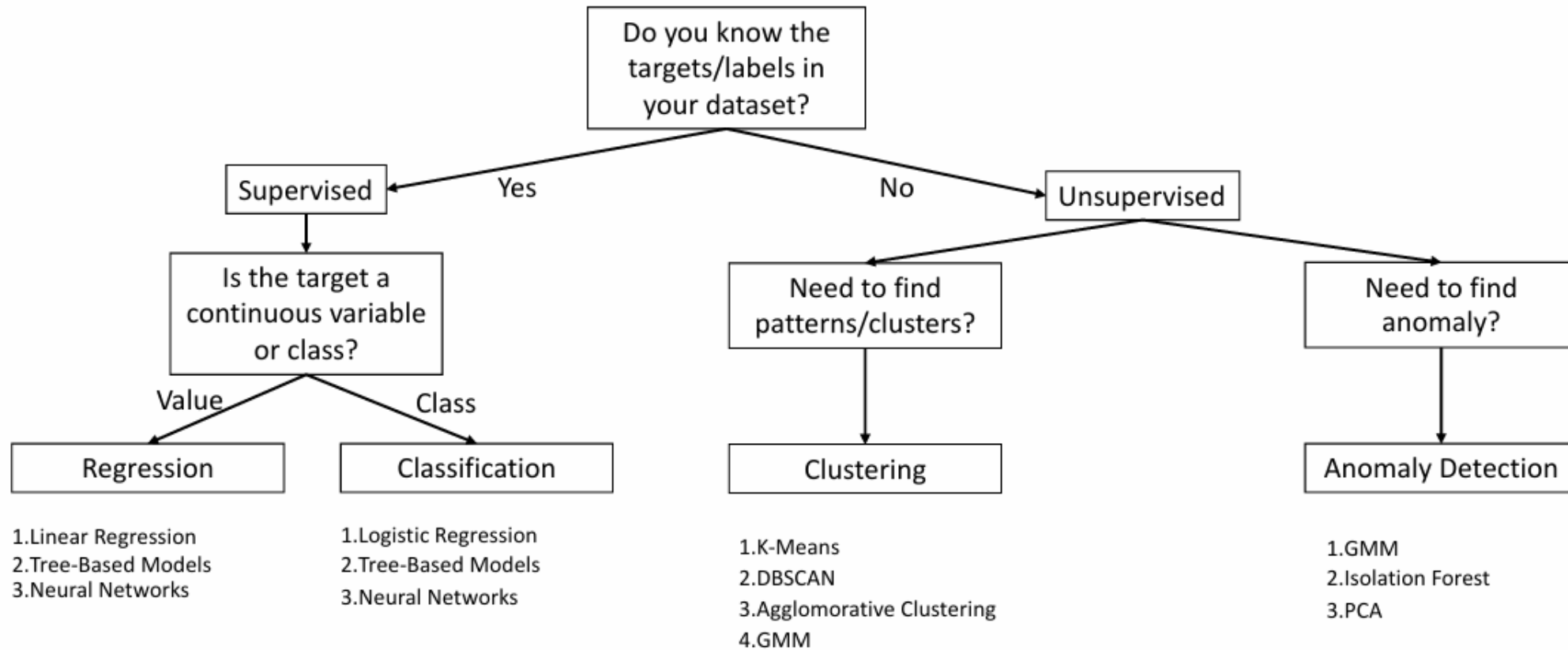
Continuous Data – mm of rainfall tomorrow

Categorical data – Will it rain or no?

Traditional Approach vs AI



Summary of Predictive AI

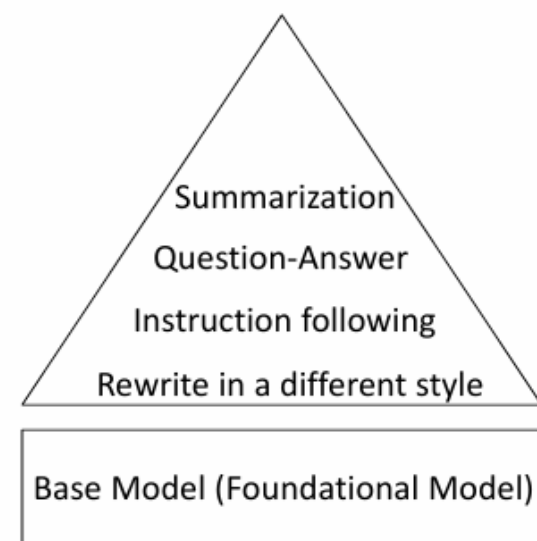


Generative AI

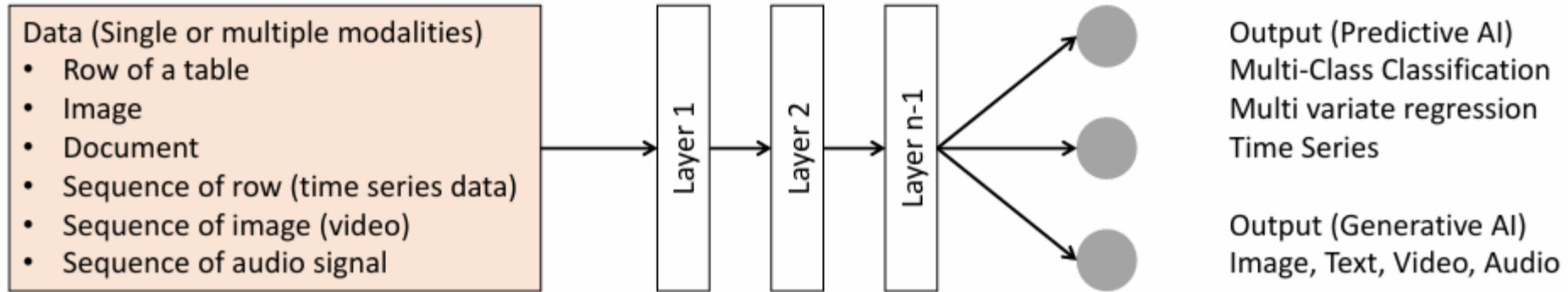
- Text to Text
 - Text to Image/Video
 - Image/Video to Text
 - Image/Video to Image/Video
 - Text to Audio
 - Audio to Text
 - Text/Image to Code
-
- Input is the “Prompt”; Model is a Large Language/Vision Model;
Output is Image/Video/Text/Speech

Foundational Models

- Large-scale AI model trained on vast amounts of diverse data
- Serves as a base for multiple downstream tasks and applications
- Key characteristics:
 - Broad knowledge and capabilities
 - Prompt engineering to make it perform tasks
 - Retrieval Augmented Generation for tapping into specific data
 - Adaptable through fine-tuning
 - Generalize to new tasks with minimal additional training
- **Examples:** GPT, BERT, T5



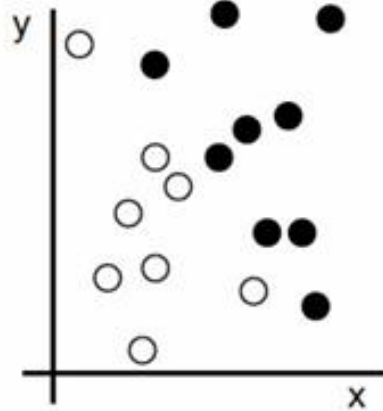
Neural Network



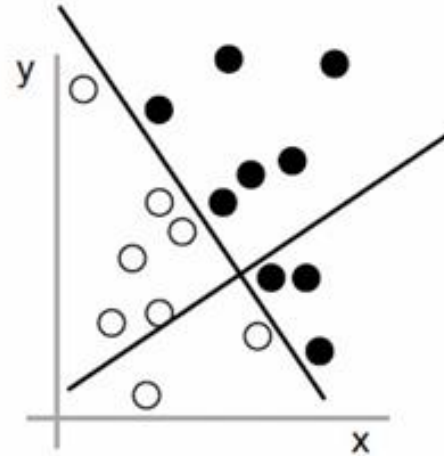
- Each Layer Consists of Neurons
- Neurons have a linear combination of input, followed by application of nonlinear activation function. Have parameters (weights and bias)
- Several Neurons form a layer
- Several Layers form a network
- Input has data
- Output layer depends on task
- Important Layers: Dense, Convolution, Multi-Head Attention, Masked MHA

Efficient Representation of Data

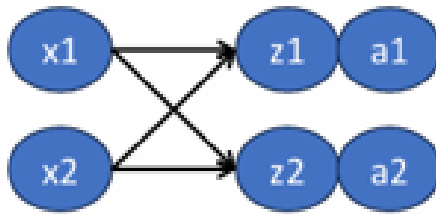
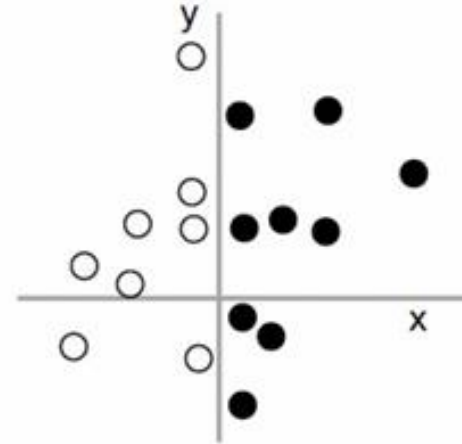
1: Raw data



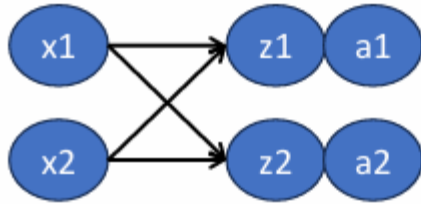
2: Coordinate change



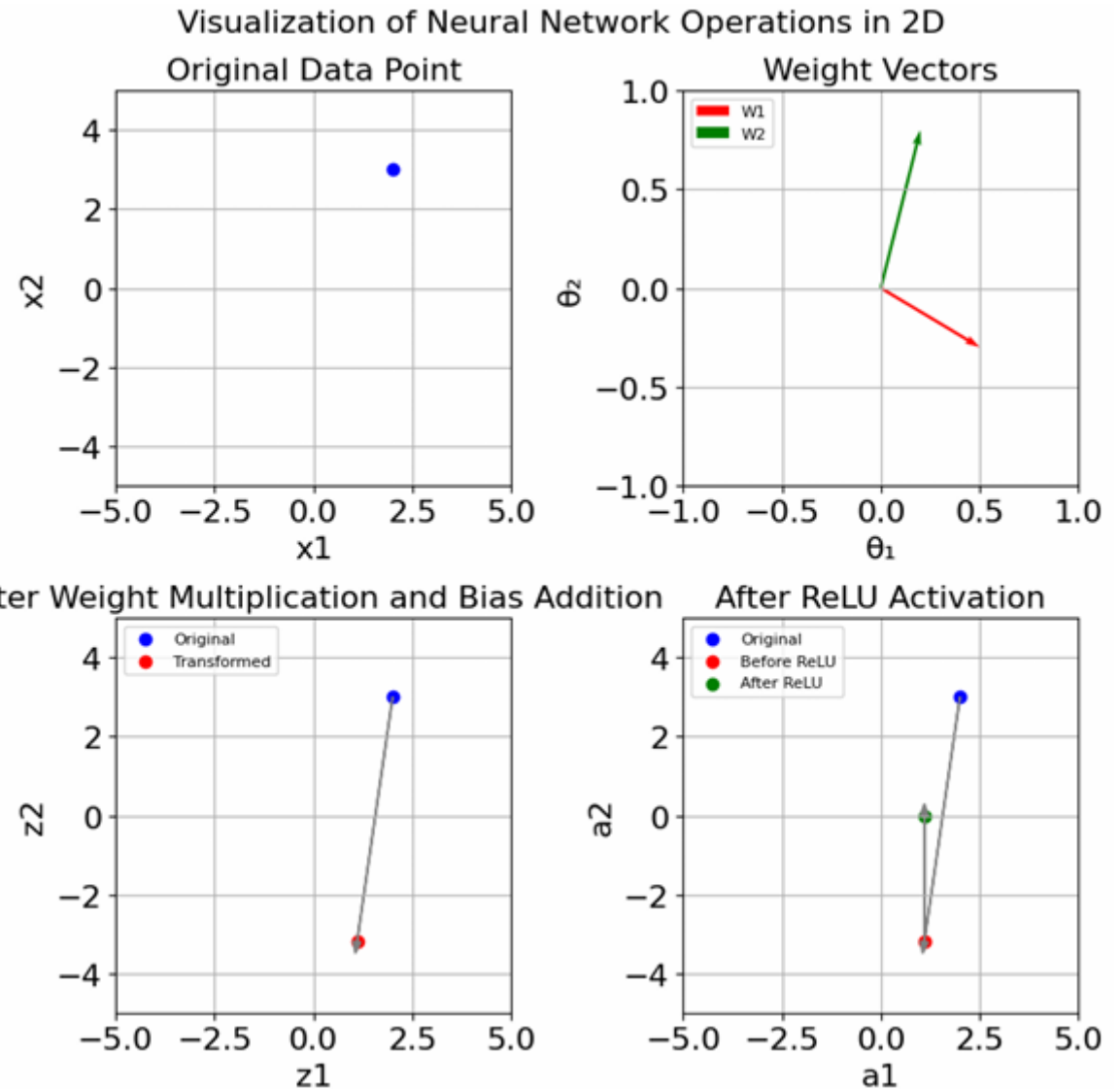
3: Better representation



Operations in a neural network



- x_1, x_2 are input neurons
- $z_1 = \theta_{1,w_1}x_1 + \theta_{2,w_1}x_2 + b_1$
- $z_2 = \theta_{1,w_2}x_1 + \theta_{2,w_2}x_2 + b_2$
- $a_1 = \max(0, z_1)$
- $a_2 = \max(0, z_2)$



What is neuron?

- Computational Unit, Data Processing Unit
- It does two activities
 - Take a weighted linear combination of input, add a bias
 - Apply an activation function on the result

Activation Functions

- Linear $a = z$
- Rectified Linear Unit (ReLU) $a = \max(0, z)$
- Sigmoid $a = \frac{1}{1+e^{-z}}$
- Softmax $a_k = \frac{e^{z_k}}{\sum_{k=1}^K e^{z_k}}$

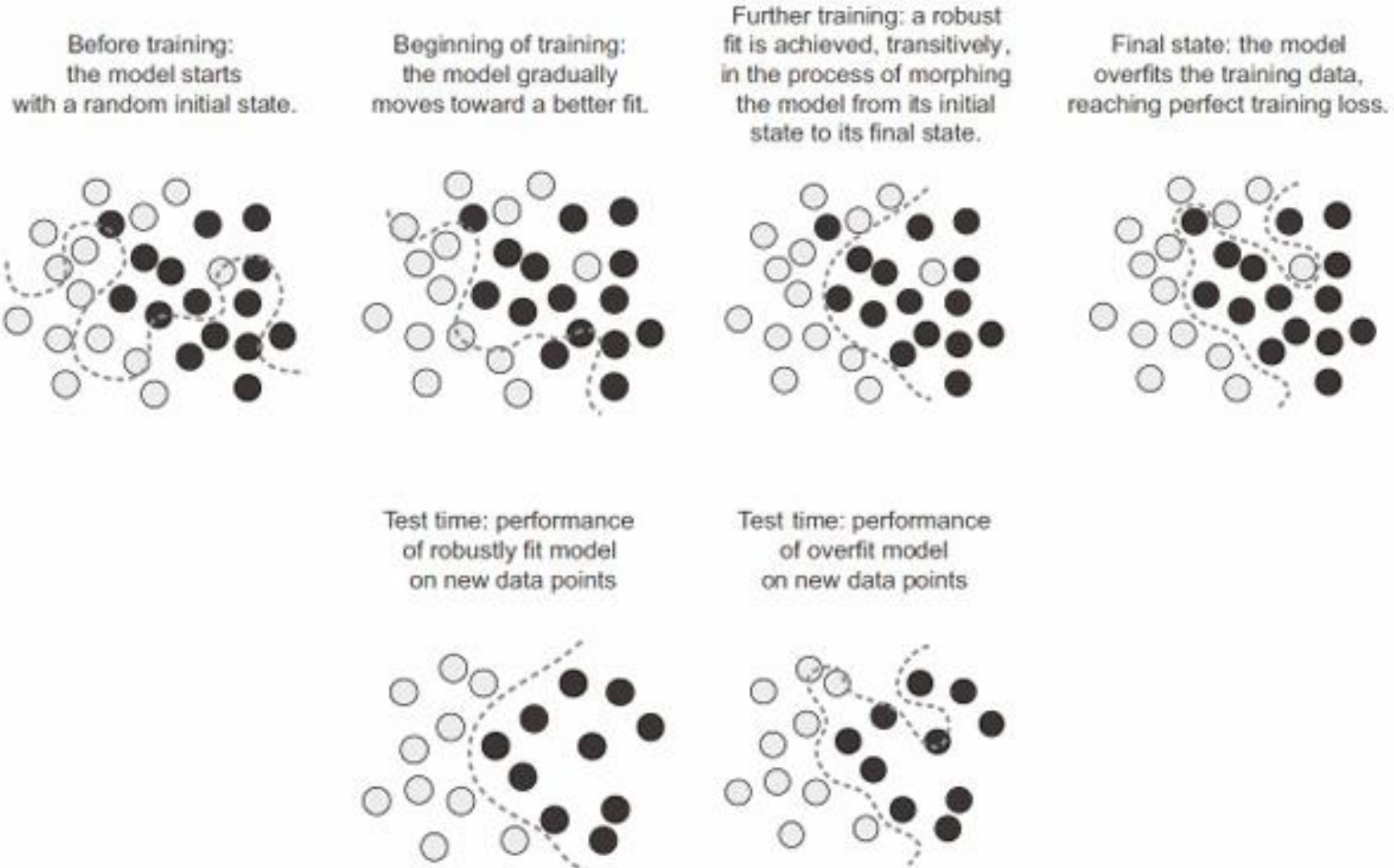
Visualizing the action of a neural network

- Neural layers transform the data to different representations
- The transformations are effected through weights and biases
- These weights and biases are learnt from data during training
- The learnt representation make the downstream task easier
- Prior to deep neural networks, these representations were hand crafted using a mechanism called as feature engineering (from domain knowledge)



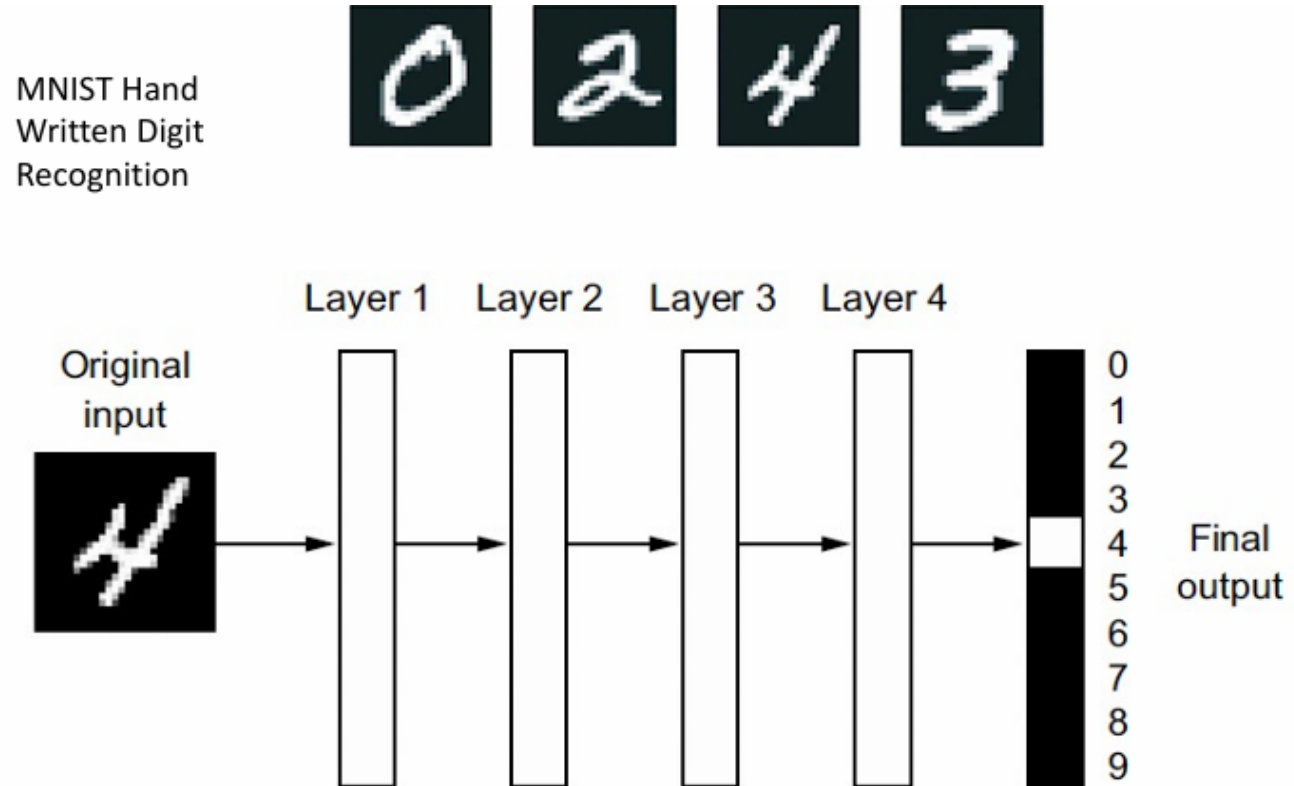
Layers of a neural model unfolds the crumbled paper ball (data in complex manifolds) to an unfolded paper (simple representation).

Decision Boundaries with Training



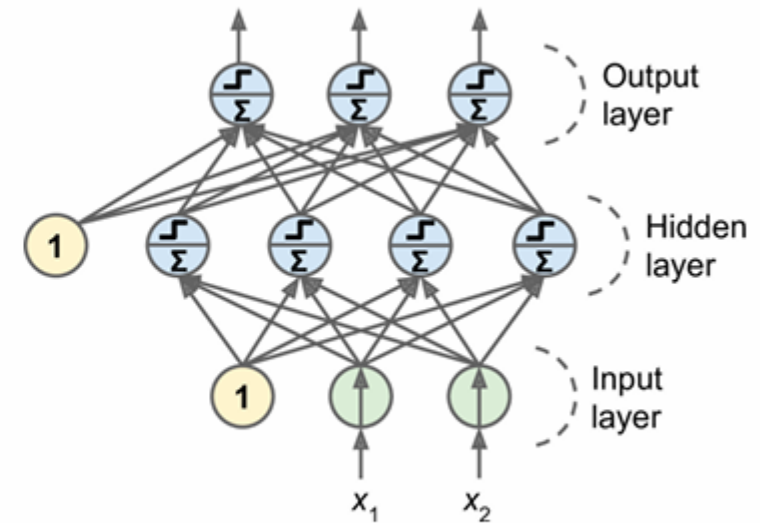
Neural Network for Classification

- 2D image is flattened to a vector
- Vector is passed through layers
- Final output layer has 10 neurons corresponding to 10 class classification

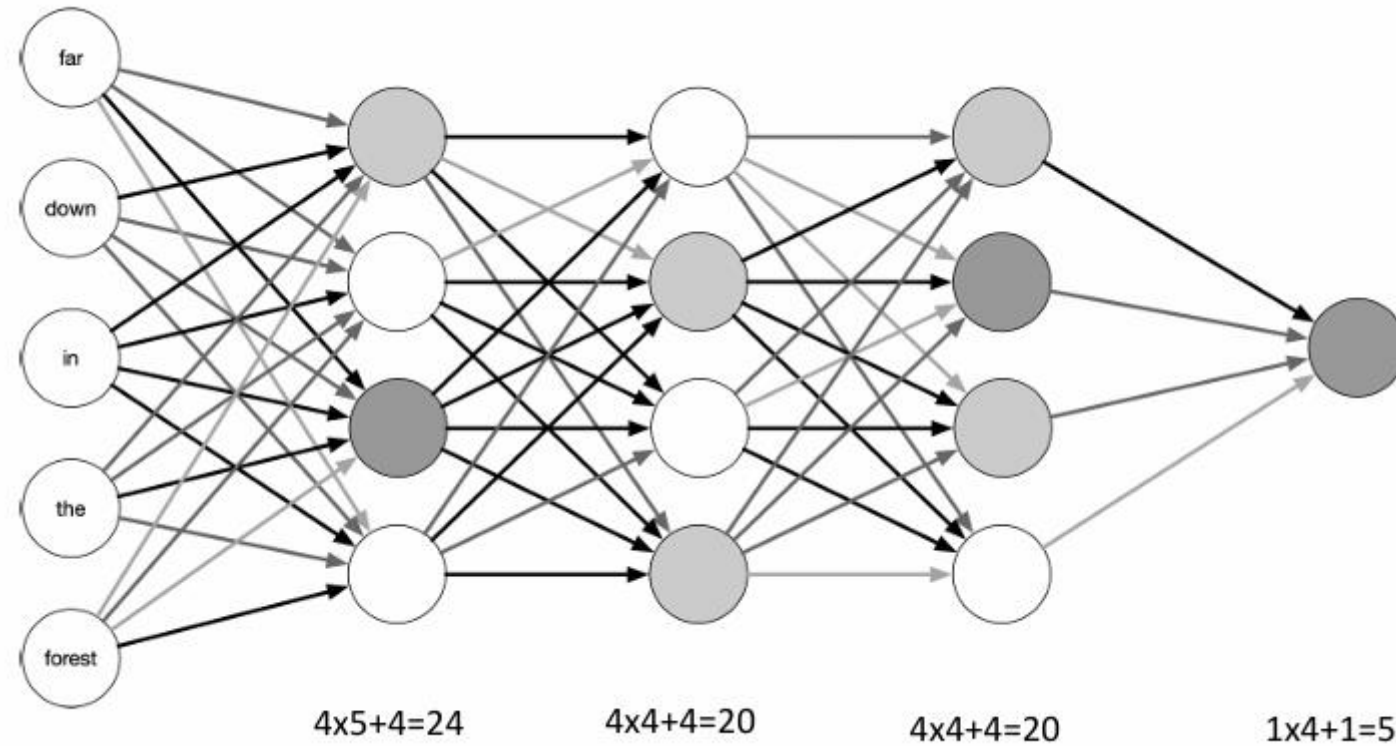


Dense Neural Network

- Input layer is a passthrough layer
- Hidden layers are composed of neurons
- Output layer returns
 - a number (as many neurons as quantities of interest) for regression tasks and
 - probability of a class for classification tasks (as many neurons as number of classes)
- Layers closer to the input are called as lower layers
- Layers closer to the output are called as upper layers
- Here the signal only flows in one direction, so this architecture is called as a feedforward neural network (FNN)

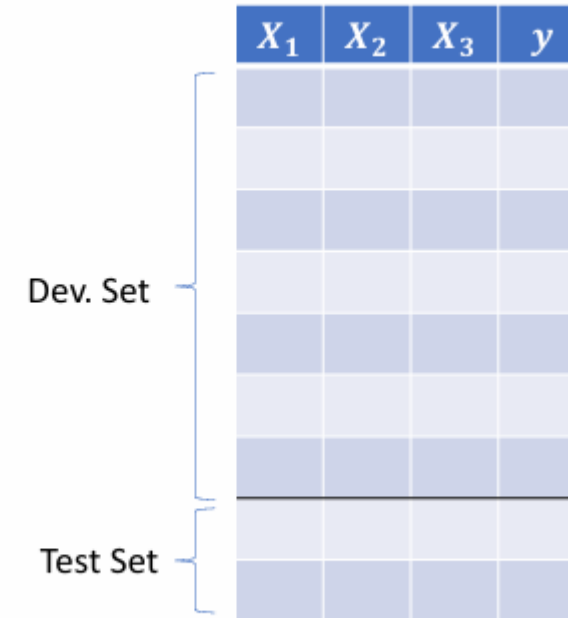


Calculating Size of a Dense Neural Network



Development-Testing Paradigm

- Consider a data set with m rows and n columns
- Development Set
 - Used to train a ML model
- Training a model involves
 - Finding parameters or growing trees
 - Uses data and an optimization algorithm working on some loss
- Development involves training and hyper-parameter tuning
 - K-Fold Cross Validation is used
- Testing involves using a developed model on totally unseen data and evaluating an expected real world performance

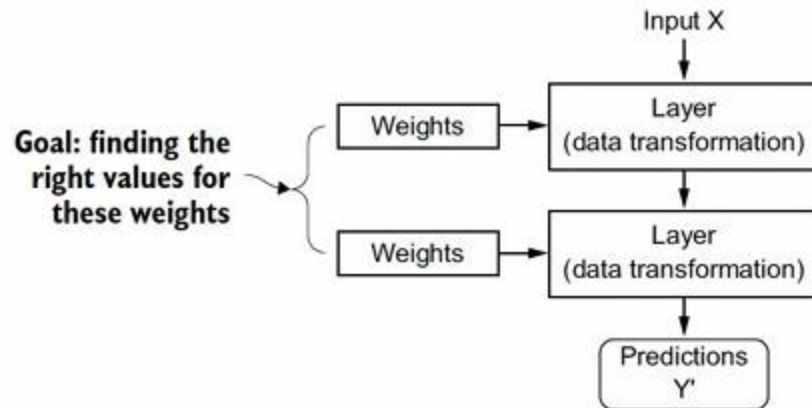


The diagram illustrates a data matrix with 10 rows and 4 columns. The columns are labeled X_1 , X_2 , X_3 , and y . The first 8 rows are grouped under the label 'Dev. Set' with a bracket on the left. The last 2 rows are grouped under the label 'Test Set' with a bracket on the left. A horizontal line separates the Dev. Set from the Test Set. The cells in the matrix are shaded in alternating light and dark blue.

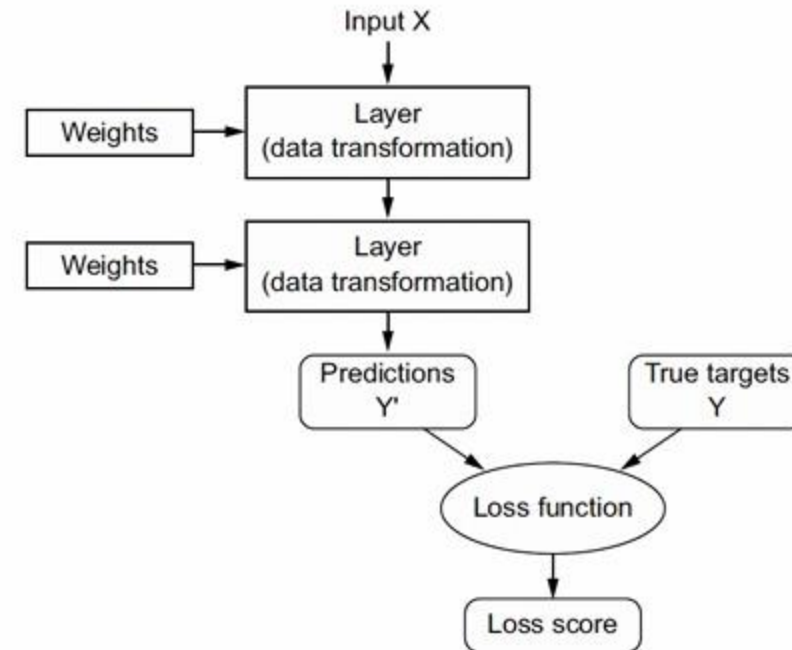
	X_1	X_2	X_3	y
Dev. Set				
Test Set				

Training the Neural Network

Goal of training is to find parameters (Weights and bias of every neuron)



Ground truth supervision data gives us the loss



Loss Function

- Regression – Mean Square Error

$$MSE = \frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i)^2$$

m – num data points
 K – num of classes

- Classification – Cross Entropy

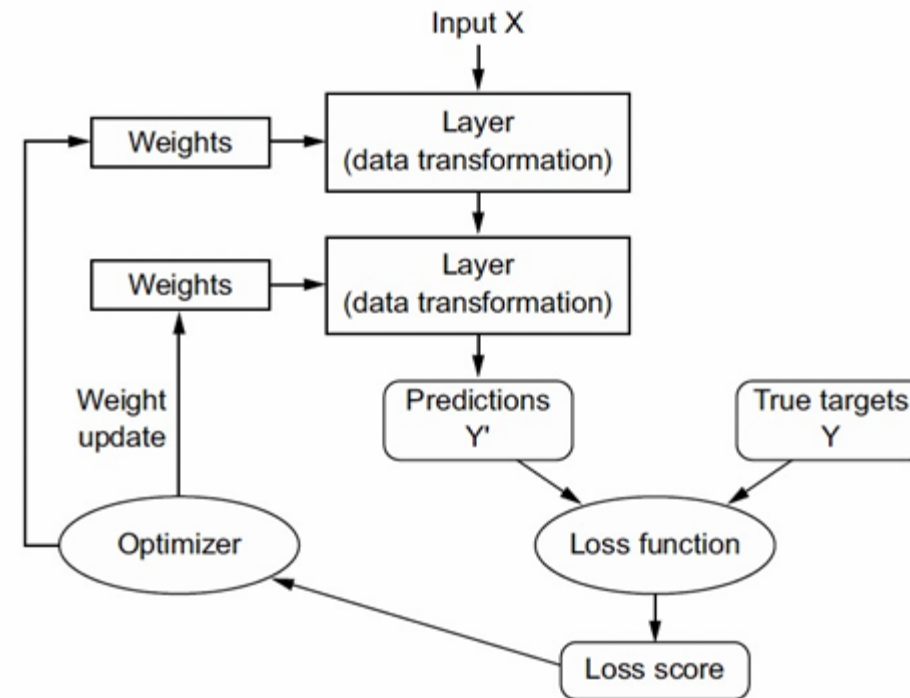
$$CE = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^K y_{i,k} \log \hat{p}_{i,k}$$

Training the Neural Network

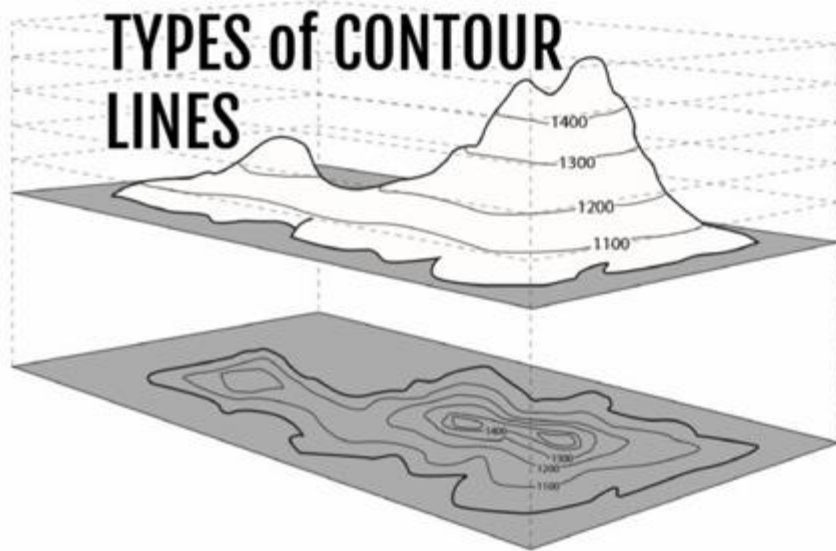
An optimizer adjusts the weights (and bias) in such a way that the loss function reduces

The process of finding these weights (and bias) is called training

A training algorithm called Stochastic Gradient Descent (variant called ADAM) is used for the training

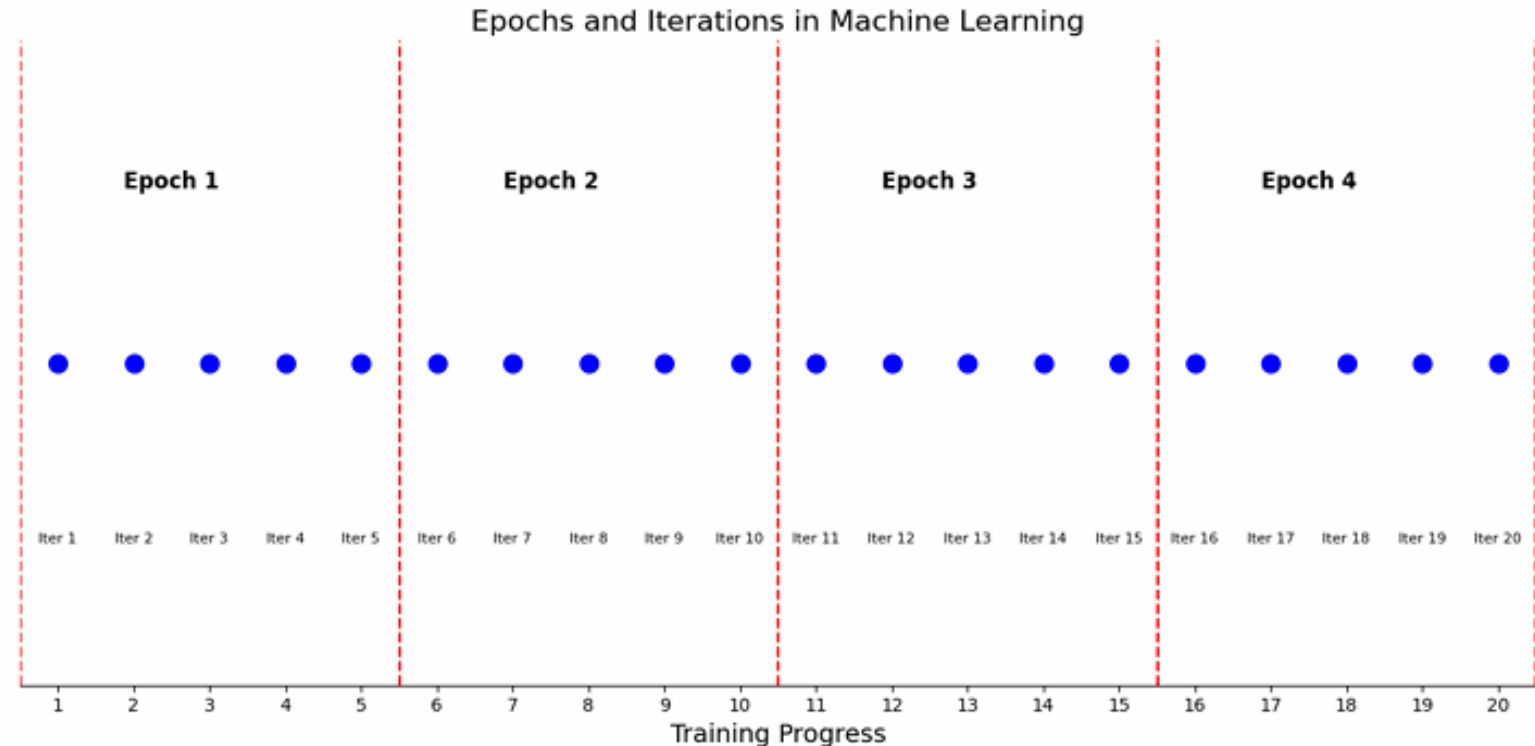


Gradient Descent: Figure Explanation

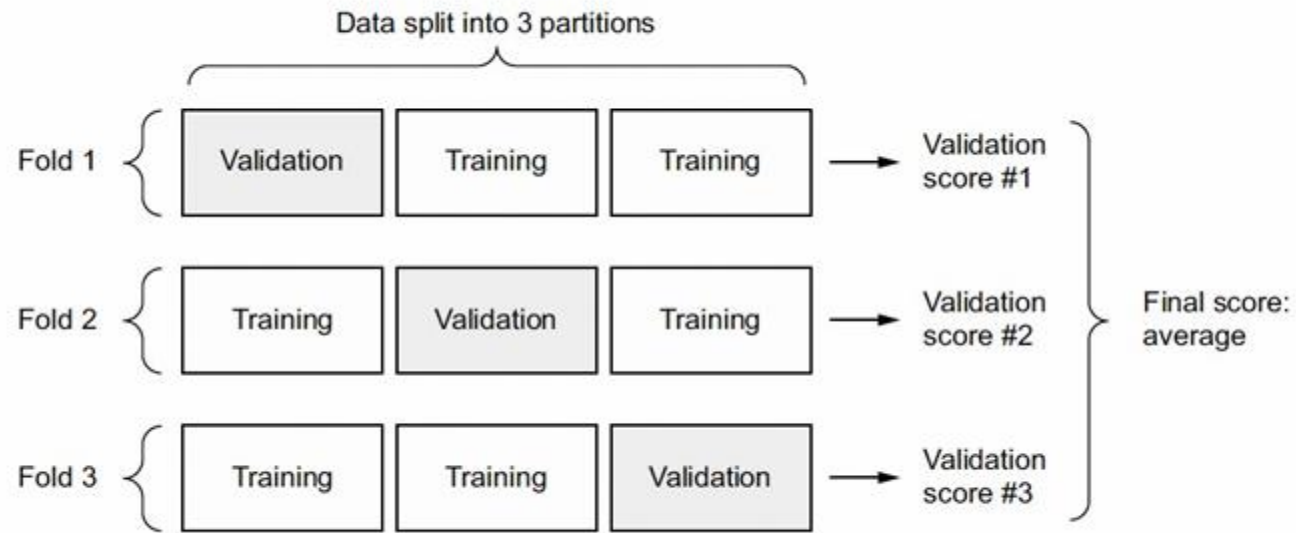


Iteration, Batch, Epoch

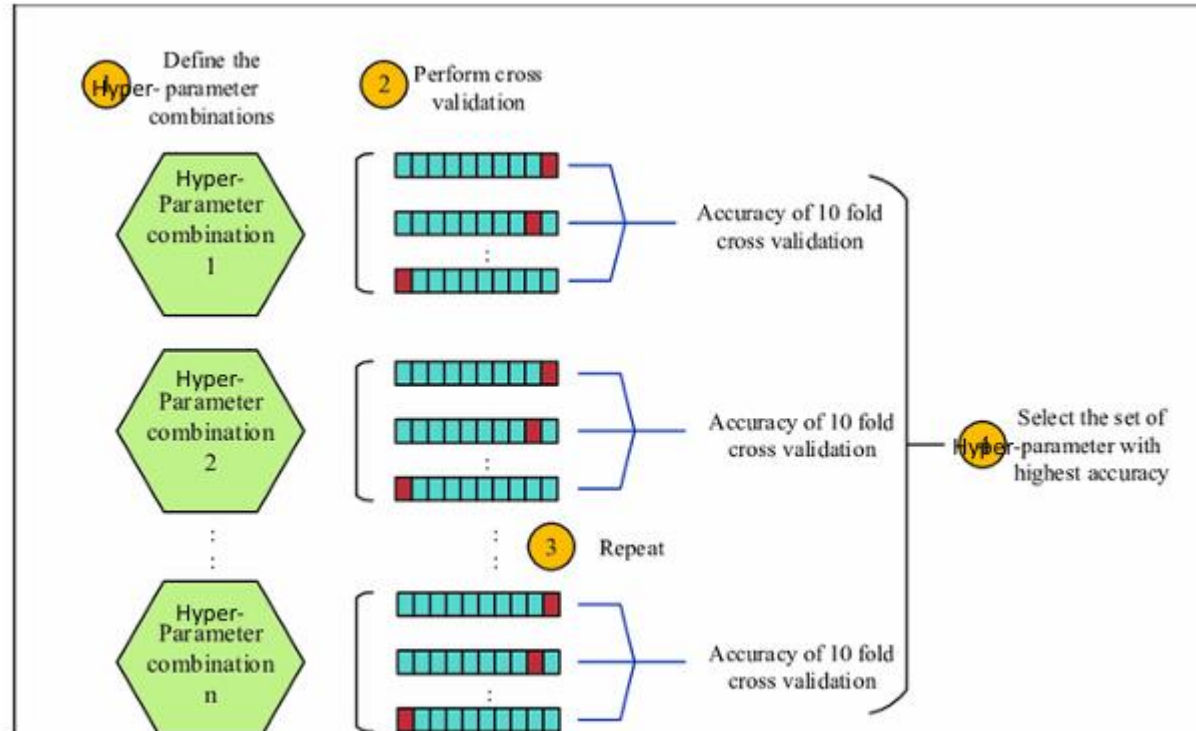
- Data Set = 1000 samples
- Batch Size = 200
- One iteration uses 200 data points to calculate loss and derivative of the loss with respect to all params
- 5 iterations comprise one epoch
- In this example 4 epochs are shown
- In each epoch all the parameters are updated 5 times (in this example)
- Epoch 2 starts with Iter 6
- It updates the parameters at the end of Iter 5 (Epoch 1)



K Fold Cross Validation



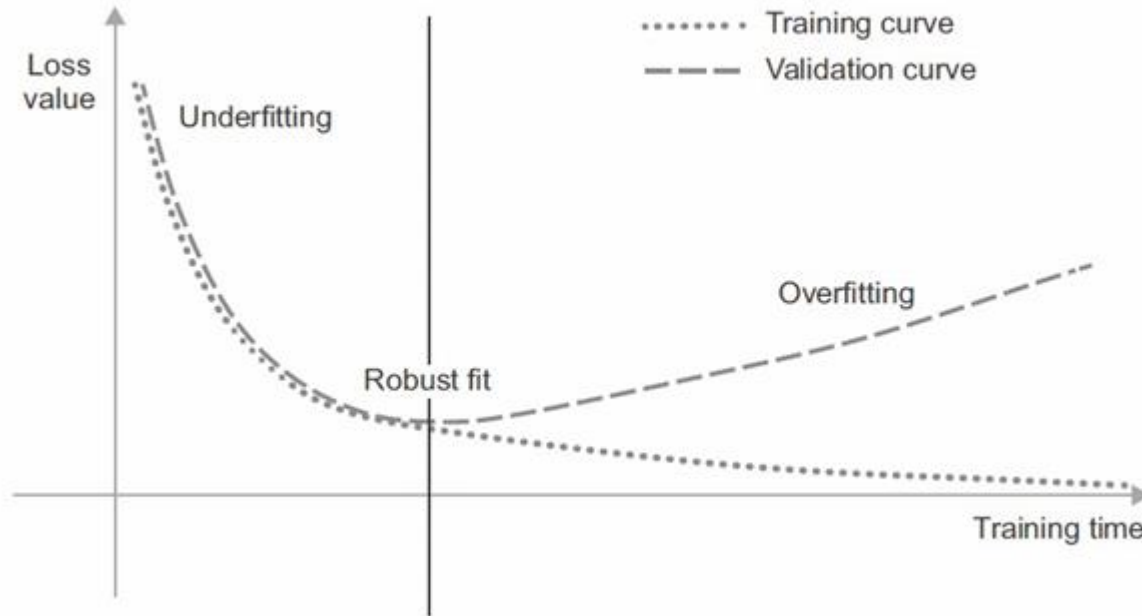
Hyper-parameter Tuning



Parameters and Hyper-parameter

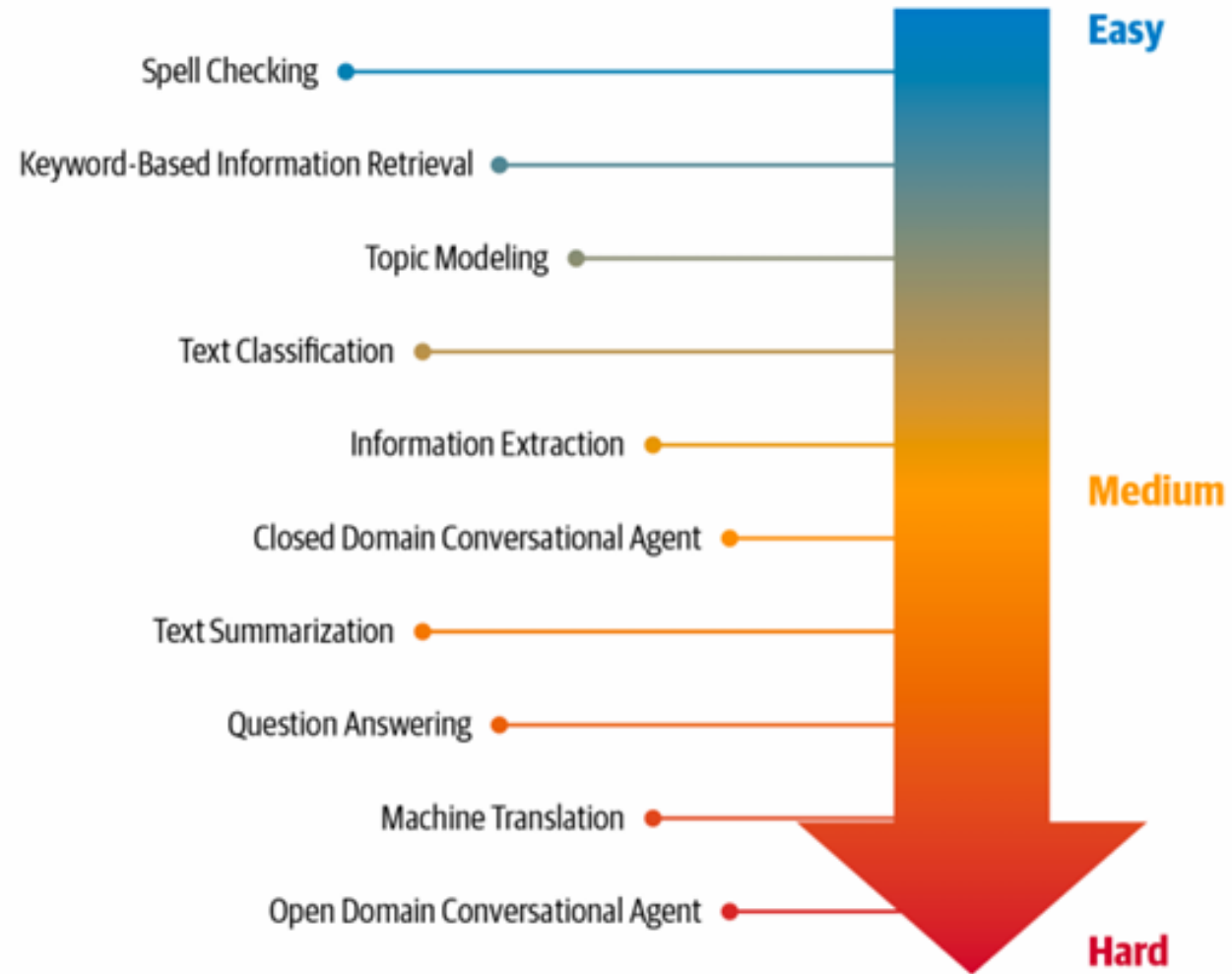
Aspect	Parameters	Hyperparameters
Definition	Variables learned from training data	Configurations set before training
Examples	Weights in neural networks, coefficients in linear regression	Learning rate, number of trees in a random forest, batch size in neural networks
Computed by	Learning algorithm (SGD, or Tree growth)	Set manually or through (brute force or optimized) search techniques (Cross validation, Grid search)
Influence	Directly affect model predictions	Affect the training process and model performance

When to stop? - Training and Validation Error

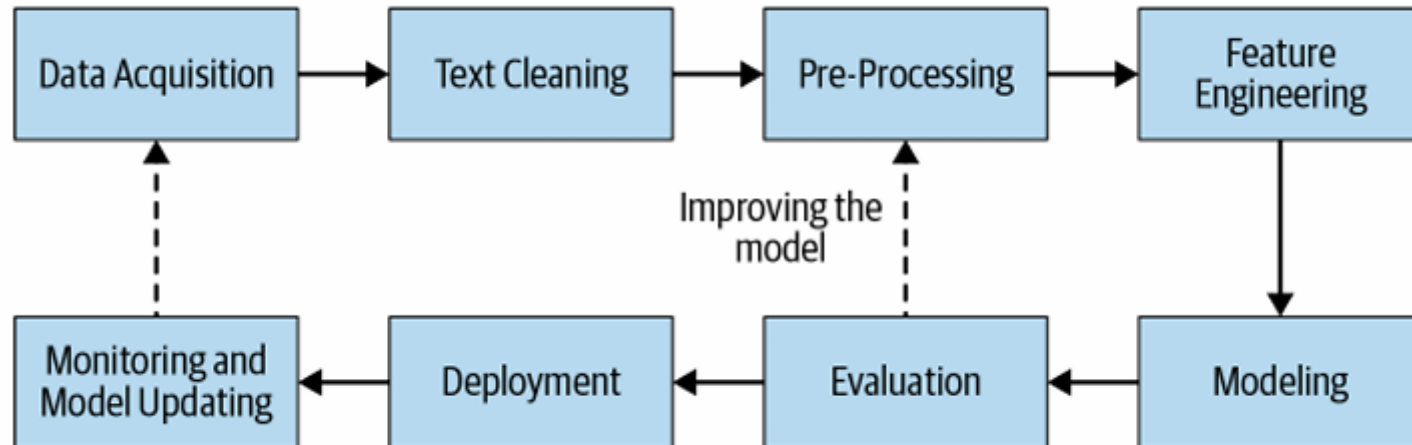


Hands-on “*Linear & Logistic Regression with Neural Network*”

NLP: Major tasks



Generic NLP Pipeline

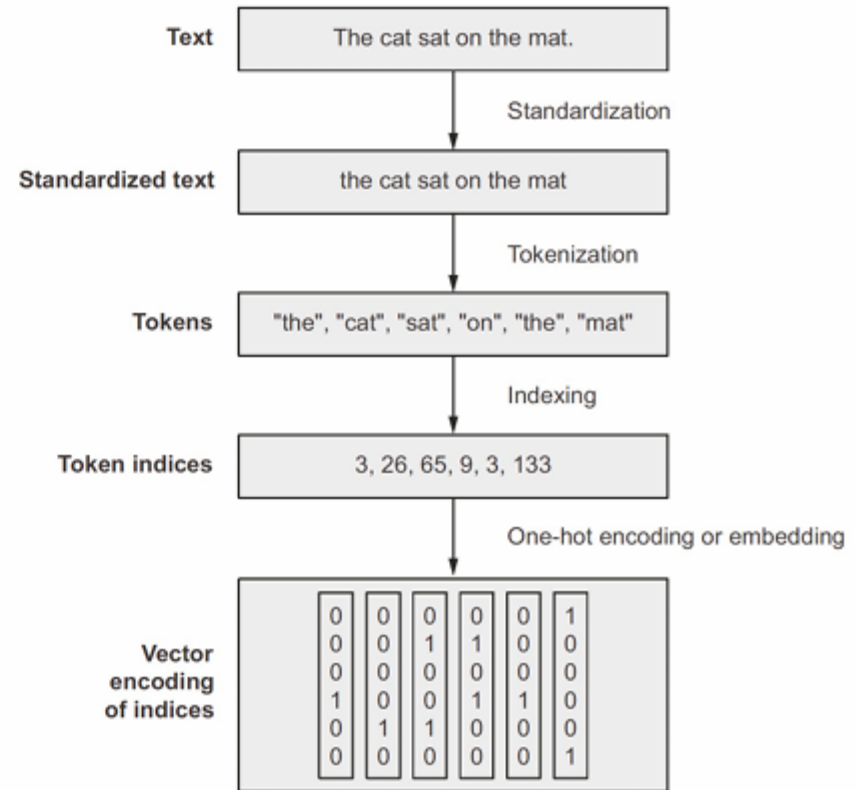


Text Cleaning

- Preliminaries (in all projects) – sentence segmentation and word tokenization
- Frequent steps (in almost all projects) - Stop word removal, stemming and lemmatization, removing digits/punctuation, lowercasing, etc.
- Other steps (in some projects) - Normalization, language detection, code-mixing (mixing of 2 or more languages), transliteration, etc.
- Advanced processing (in specific applications) – POS tagging, parsing, coreference resolution

Data Pre-Processing for NLP

- DL Models are differentiable functions
- They take only numeric tensors as input – Cannot handle raw text
- Text Vectorization – Process of converting text to numbers is crucial to success of NLP
- Template for text vectorization
 - Standardize
 - Remove punctuation, convert to lowercase etc
 - Tokenize
 - Split text into units (tokens) – can be character, words, groups, sentences etc
 - Index
 - Given a numerical value to the tokens
- Representation of text in numerical vector form is also called as embedding



Tokenization

- Definition: Process of breaking down text into smaller units (tokens)
- Purpose: Convert text into a format processable by the model
- Importance:
 - Fundamental to natural language processing (NLP)
 - Crucial for model input in generative text models (e.g., GPT)
- Tokens can represent:
 - Words
 - Parts of words (subwords)
 - Individual characters
- Impact on model performance:
 - Determines vocabulary size
 - Affects language handling capabilities
 - Influences understanding of context and meaning
- Key considerations:
 - Efficiency of text processing
 - Preservation of semantic and syntactic information

Example

- Original text: The quick brown fox jumps over the lazy dog.
- Tokens: ['the', 'quick', 'bro', 'wn', 'fox', 'jump', 's', 'over', 'the', 'lazy', 'dog', '.']
- Visualized tokenization: [the] [quick]
[bro][wn] [fox] [jump][s] [over] [the] [lazy]
[dog][.]

Byte Pair Encoding for Vocabulary Generation

- GPT models use BPE
- Start with individual characters represented by UTF-8, that is 256 tokens
- Make new tokens to be included in the vocabulary by merging pairs of tokens that are most frequent
- Repeat until desired vocabulary size is achieved
- Advantages:
 - Handles out-of-vocabulary words well
 - Balances common words and subwords
 - Can capture meaningful subword units (e.g., prefixes, suffixes)
- Considerations:
 - Vocabulary size affects granularity of tokenization
 - May create unintuitive subword splits
 - Requires retraining for new domains or languages

Vocabulary Indexing and Text Vectorization

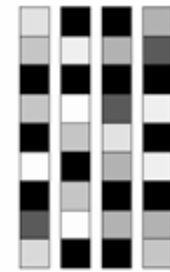
- To encode split tokens into numerical vectors, we use vocabulary indexing
 - Use a vocabulary and assign a unique number to that vocabulary
 - Vocabulary size is a hyperparameter
 - Do a one hot encoding of tokens
 - Map the one-hot encoding to a text vector to an embedding dimension
 - Embed_dim is a hyperparameter
- Advantages:
 - Allows all words to be represented uniquely across language tasks
 - It opens up possibilities of learning word vectors that have a “mathematical distance” between them that preserves meaningful relationships as understood by humans!

Word Embedding

- One-Hot encoding of word vector makes an orthogonal words assumption
- This encoding considers that all words are independent of each other
- For words, this is a wrong assumption
- “Closet”, “Cupboard”, “Toilet” are not orthogonal words
 - Closet and Cupboard should be close or same vector in American English
 - Closet and Toilet should be close of same vector in Indian English
- What we want? – Geometric relationship between words should reflect the semantic relationship
 - L2 distance or cosine distance between words must reflect their “Semantic distance”
- Word Embeddings – vector representation of words that map human language into a structured geometric space

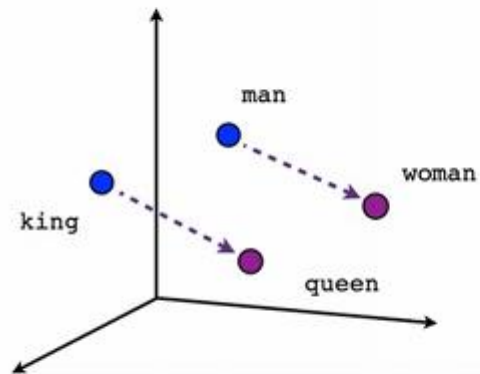


One-hot word vectors:
- Sparse
- High-dimensional
- Hardcoded

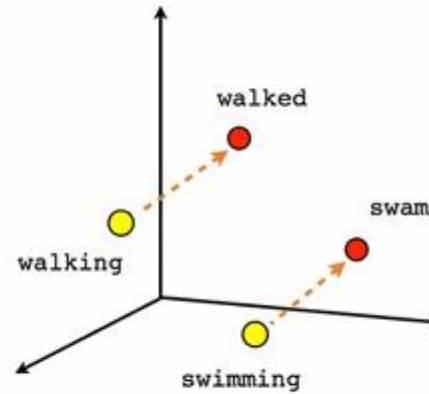


Word embeddings:
- Dense
- Lower-dimensional
- Learned from data

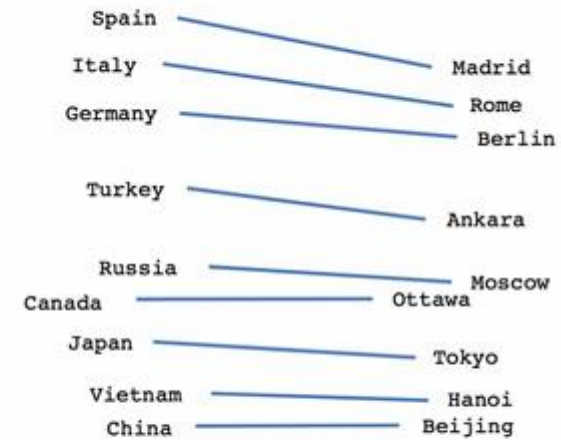
Visualization of Word Vectors



Male-Female



Verb tense



Country-Capital

Embedding Layer

- Learn word embeddings jointly with the main task you care about
- Initialize to random word vectors (map)
- The embedding of a single word vector is a matrix vector multiplication
- The embedding matrix is learnt via backpropagation

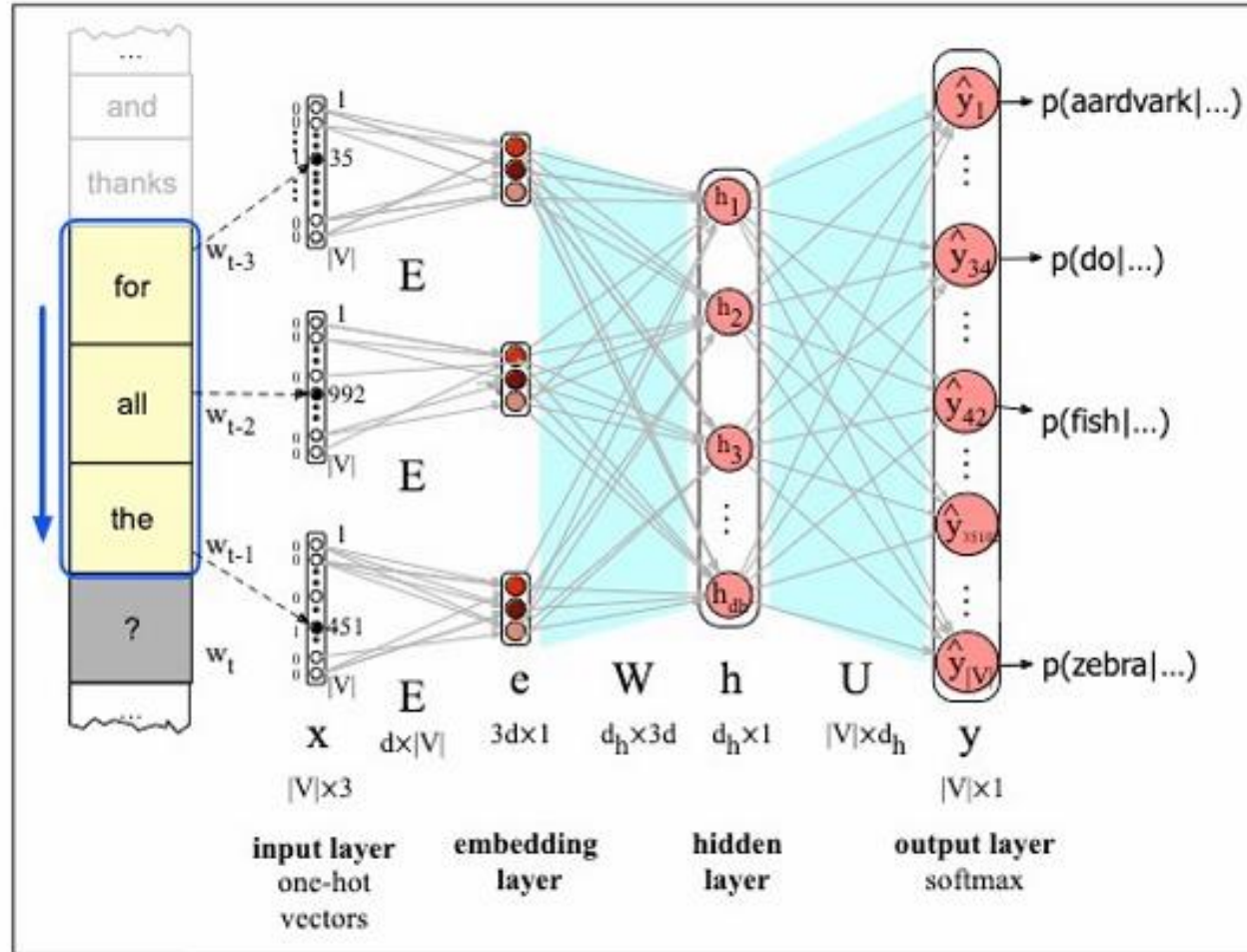
```
embedding_layer = layers.Embedding(input_dim=max_tokens, output_dim=256) ←
```

The Embedding layer takes at least two arguments: the number of possible tokens and the dimensionality of the embeddings (here, 256).

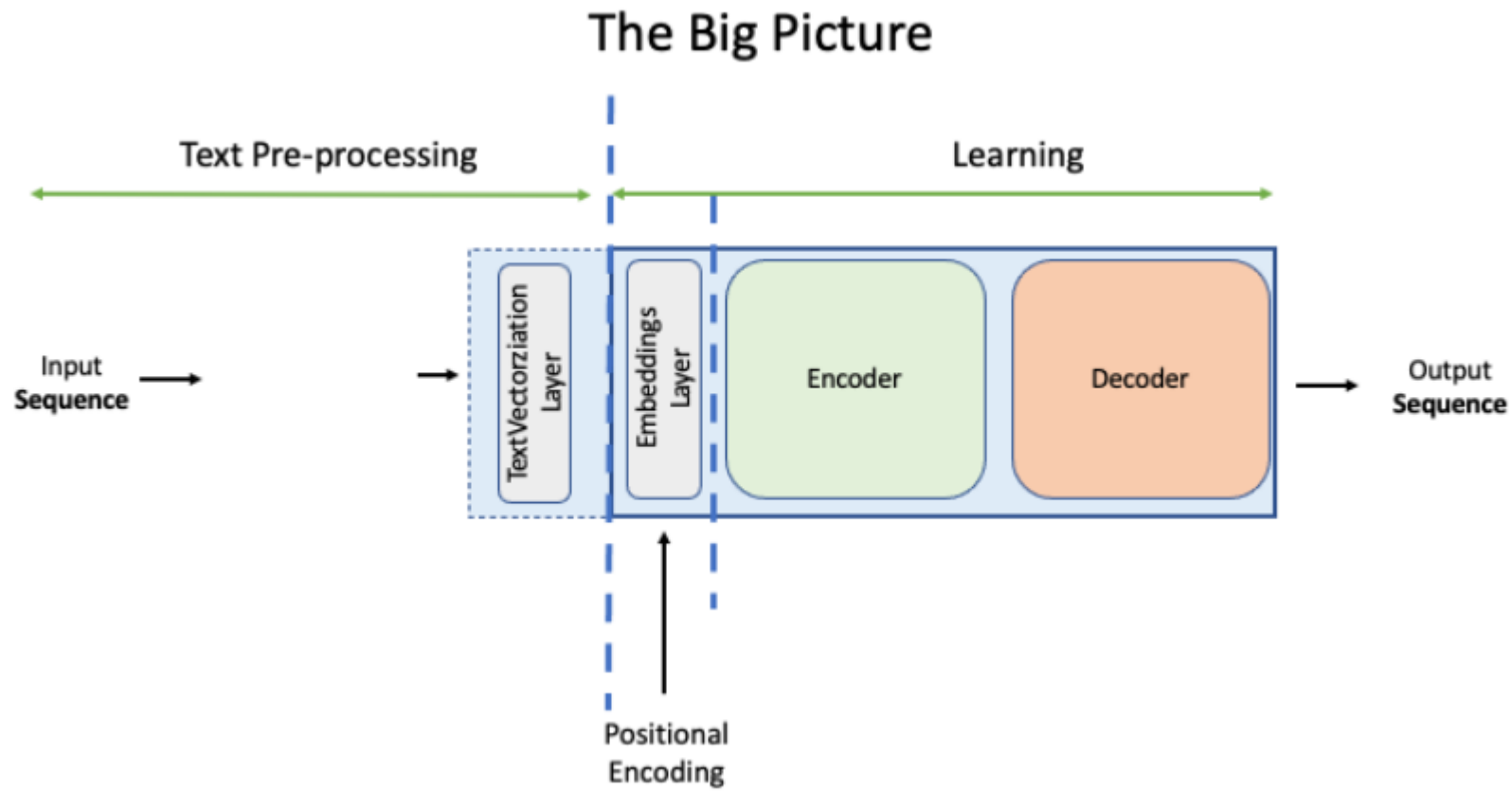
Word index → Embedding layer → Corresponding word vector

- Pre-trained embeddings can also be used such as Google Word2Vec, GloVe

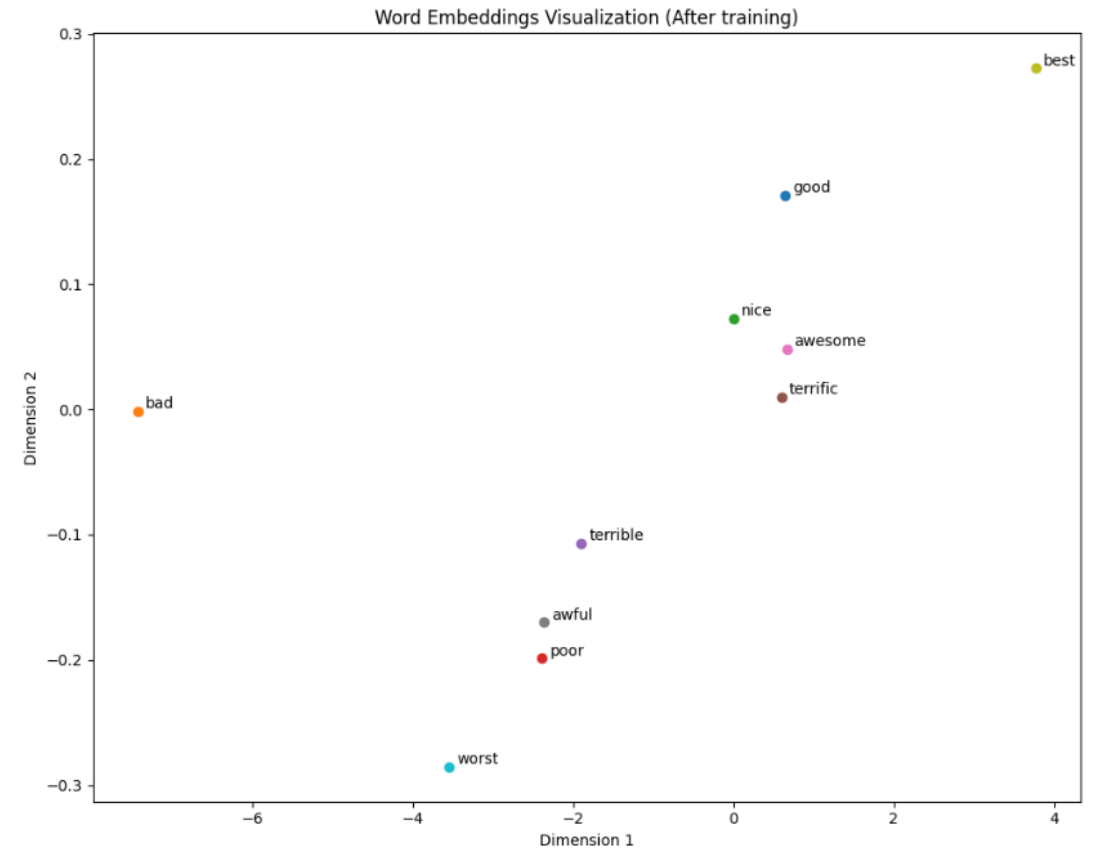
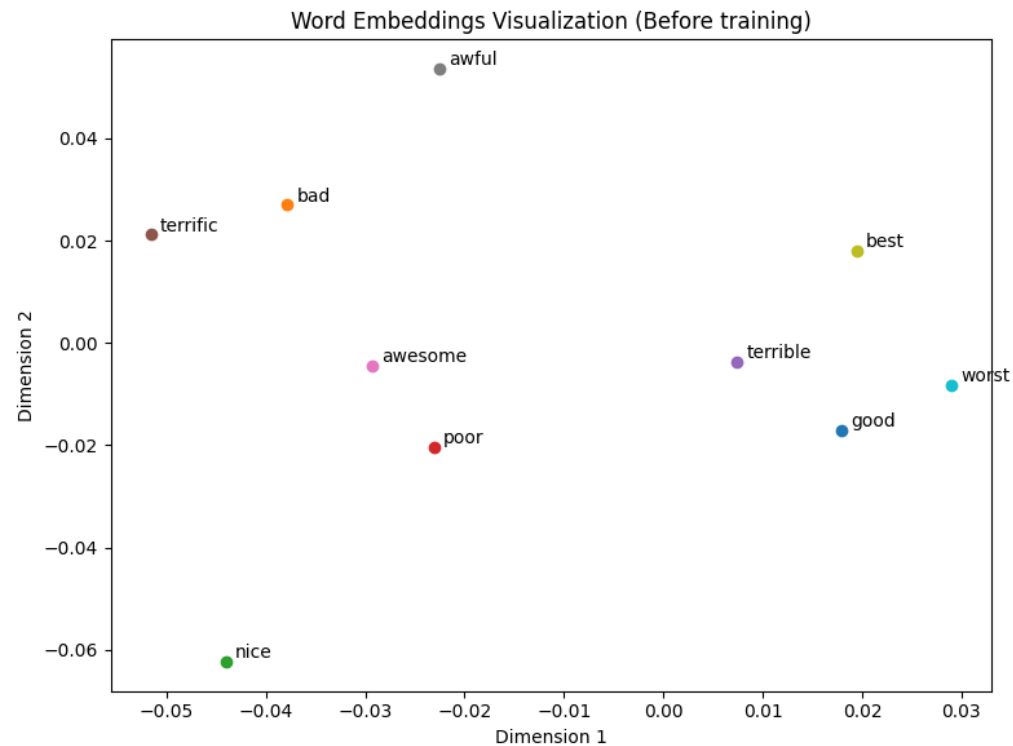
Word Embedding – Pictorial Explanation



Hands-on “*Text Vectorization & Word Embedding Experiment*”



Hands-on “*Text Vectorization & Word Embedding Experiment*”





Thank you!